

Replit Prompt: Dropbox Storage Implementation

Paste verbatim. Adobe Firefly take-home, creative automation pipeline.

Implement Dropbox cloud storage support as the primary cloud adapter for the StorageAdapter pattern. The PDF's Data Sources section names Azure, AWS, or Dropbox as the expected cloud storage backend. We previously scaffolded the StorageAdapter interface with LocalStorageAdapter implemented and S3/Azure/Dropbox as NotImplementedError stubs. This task implements the Dropbox adapter end-to-end and updates the run manifest, logging, and UI to reflect cloud storage usage.

Strategic context (for code comments and README content):

Dropbox was chosen over S3 and Azure because the customer's workflow is creative team-led. Creative and marketing teams at consumer goods companies consume assets in Dropbox-class shared storage, not in S3 consoles or Azure Blob containers. S3 and Azure remain scaffolded as single-class additions for engineering-side observability or hybrid dual-write architectures. The chosen adapter reflects user workflow, not just technical preference. Capture this rationale in inline code comments where the adapter is selected and in the README content prepared at the end of this task.

Implementation:

Install the official Dropbox SDK: `npm install dropbox`

Build out the DropboxStorageAdapter class that currently throws NotImplementedError. Implement against the existing StorageAdapter interface.

Required methods:

save(path, content, metadata?): Upload to Dropbox using filesUpload with mode overwrite. After upload, create a shared link via sharingCreateSharedLinkWithSettings. Return { url, bytes } where url is the shared link and bytes is content size.

load(path): Download from Dropbox using filesDownload. Return as Buffer.

exists(path): Use filesGetMetadata wrapped in try/catch. Return true if metadata returned, false on path/not_found error.

getUrl(path): Check for existing shared link first via sharingListSharedLinks. If none exists, create one via sharingCreateSharedLinkWithSettings. Return the public URL.

adapterName: Return string "dropbox"

Authentication:

Read access token from DROPBOX_ACCESS_TOKEN env var. If missing when the adapter is instantiated, throw a clear error message:

"Dropbox adapter requires DROPBOX_ACCESS_TOKEN environment variable. Generate a token at <https://www.dropbox.com/developers/apps>"

Create a .env.example file at the repo root with:

```
# Storage adapter selection
STORAGE_ADAPTER=local

# Set to "dropbox" and provide token to use Dropbox storage
# STORAGE_ADAPTER=dropbox
# DROPBOX_ACCESS_TOKEN=your_token_here
```

Path structure:

Maintain the same path structure as local: /outputs/{client}/{product}/{ratio}/{filename}. Dropbox stores this as a nested folder hierarchy. Client names with spaces should be slugified (lowercase, underscores) for path safety.

Retry behavior:

Apply the existing retry utility (built for Nano Banana) to Dropbox filesUpload and sharingCreateSharedLinkWithSettings calls. Same rules: 3 attempts max, exponential backoff with jitter (1s, 2s, 4s with plus or minus 20 percent jitter), retry on 429/500/502/503/504 and network errors, do not retry on 400/401/403.

Error handling:

Wrap all Dropbox API calls in try/catch. On failure:

- Log structured error with operation, path, error type, error message
- Do not crash the pipeline. Log the failure and continue processing remaining assets.
- If save fails after all retries, the run manifest marks that asset as uploadStatus: "failed" with error details attached.
- Successful uploads get uploadStatus: "success" and the shared URL.

Structured logging:

Every Dropbox operation produces a log entry. Success format:

```
{
  "agent": "organizer",
  "adapter": "dropbox",
  "operation": "save",
  "path": "/outputs/vitara_naturals/energize_serum/1x1/creative.jpg",
  "bytes": 245678,
  "latencyMs": 1234,
  "status": "success",
  "url": "https://www.dropbox.com/s/...",
  "timestamp": "..."}
}
```

Failure format:

```
{
  "agent": "organizer",
  "adapter": "dropbox",
  "operation": "save",
  "path": "/outputs/vitara_naturals/energize_serum/1x1/creative.jpg",
  "status": "failed",
  "errorType": "rate_limit",
  "errorMessage": "...",
  "attemptsExhausted": 3,
  "timestamp": "..."}
}
```

Run manifest update:

The run manifest currently includes `localPath` and `url` per asset. Expand the per-asset record to:

```
{
  "localPath": "...",
  "url": "https://www.dropbox.com/s/...",
  "adapter": "dropbox",
  "uploadStatus": "success",
  "uploadedAt": "...",
  "bytes": 245678
}
```

The `adapter` field makes the manifest self-describing across storage backends. When running with local storage, `adapter` is "local" and `url` is the local path.

Adapter factory update:

The `getStorageAdapter()` factory currently handles "local" and throws `NotImplementedError` for cloud adapters. Update it to:

1. Read `STORAGE_ADAPTER` env var (default "local")
2. For "dropbox": instantiate `DropboxStorageAdapter`, which will throw if `DROPBOX_ACCESS_TOKEN` is missing
3. For "s3" and "azure": continue to throw `NotImplementedError` with a message indicating which adapters are implemented vs scaffolded ("Implemented: local, dropbox. Scaffolded for future: s3, azure.")
4. Log the selected adapter at run start (existing behavior)

UI updates:

In the results gallery, if cloud storage is active, the download link should point to the cloud URL instead of the local path. Detect via the manifest's `adapter` field per asset. Keep the visual styling identical. Just make sure the link target is correct regardless of backend.

Add a small adapter badge in the pipeline status section showing which storage backend was used for the run. Example: "Storage: Dropbox" or "Storage: Local filesystem". This is visible in the demo and the screenshot.

Testing flow:

After implementation, run two test campaigns to verify both paths work:

Test 1, Cloud path:

1. Set STORAGE_ADAPTER=dropbox and DROPBOX_ACCESS_TOKEN=token in Replit Secrets
2. Run pipeline with Vitara Naturals brief
3. Confirm assets land in Dropbox at the expected paths
4. Confirm run manifest includes Dropbox URLs and adapter: "dropbox"
5. Confirm structured logs show successful Dropbox save operations
6. Confirm UI gallery download links resolve to Dropbox URLs

Test 2, Local path regression:

1. Set STORAGE_ADAPTER=local (or remove env var entirely)
2. Run pipeline with the same brief
3. Confirm assets land in ./outputs/... as before
4. Confirm run manifest shows adapter: "local" and local paths
5. Confirm UI gallery still loads correctly with local paths

Show me the structured log output and run manifest JSON from both test runs so I can verify both adapters work cleanly.

Important constraint:

This is a strict additive change. The LocalStorageAdapter behavior must remain identical. The default with no env vars set must produce the same output as before this change. Cloud storage is opt-in via env var only.

README content preparation:

After implementation, prepare a "Storage" subsection under "Key Design Decisions" for the README. The section should explain: (1) the StorageAdapter abstraction with three backends supported architecturally (local, Dropbox, S3 stub, Azure stub), (2) why Dropbox was chosen for this POC over S3 and Azure (customer workflow alignment — creative teams live in Dropbox-class tools, not in S3 consoles), (3) how to switch between local and cloud via env var, (4) why local is the default in the demo video (stability during recording) while cloud is documented and verified working. Do not write the README yet. Just have this content ready to copy in.